

Gateway Evidence Services — Operator Manual

What this technology does

The gateway-evidence subsystem preserves and verifies an independent source lineage beside the ordinary application path. It answers a different question from ChirpStack and Node-RED:

Do the retained gateway journal, cloud MQTT witness, application event, raw bytes, trusted decode, and stored telemetry all agree?

It consists of separate least-privilege services. No one cloud evidence service is allowed to own the entire trust chain.

Current commissioned placement

Gateway-01

gateway-integrity-journal

gateway-journal-uploader

ULC-01

ingest-1

collector-1

fabric-adapter-1 ENABLED

ULC-02

ingest-2

verifier-1 + trusted decoder

fabric-adapter-2 DISABLED

ULC-03

collector-2

verifier-2 + trusted decoder

Cloud evidence processing is replicated:

ingest-1/2

active/active

collector-1/2

active/active; each maintains sessions to BOTH broker backends

verifier-1/2

active/active; PostgreSQL lease-fenced workers

trusted decoder

identical immutable package on both verifier replicas

raw objects

SeaweedFS cross-host durable store

metadata/state

PostgreSQL/Patroni

The physical gateway itself remains one device; replicated cloud processing does not make the radio gateway physically redundant.

Trust and authority split

Component	Authority
gateway journal	append local source evidence
uploader	upload closed evidence/checkpoints; no history rewrite

Component	Authority
ingest	authenticate upload and append metadata/raw object
MQTT collector	read-only witness of gateway MQTT events
verifier	correlate evidence and write verifier-owned result
trusted decoder	independent deterministic decode
Node-RED	application telemetry writer only
Fabric adapter	restricted outbox/OpenBao/Fabric authority only

A verifier must not possess OpenBao signing authority or a Fabric private identity.

Current deployment layout

Tracked bundle:

```
evidence-services/cloud/deploy/
compose.yml
compose.collector-mtls.yml
compose.fabric-adapter-enabled.yml
preflight.sh
fabric-adapter-enable-preflight.sh
release.commissioned.env
```

Protected runtime material:

```
/etc/lorawan-cloud/gateway-evidence/
/etc/lorawan-pki/gateway-evidence/
```

Every commissioned cloud evidence container runs as numeric 65532:65532, read-only root filesystem, capabilities dropped, no-new-privileges, and bounded PID/memory/CPU/log settings.

Step 1 — Check intended service placement

Run on each cloud host:

```
sudo docker ps --filter 'label=com.docker.compose.project=lorawan-gateway-evidence' --format 'table
{{.Names}}\t{{.Label "com.docker.compose.service"}}\t{{.Status}}\t{{.Image}}'
```

Expected logical placement:

- ULC-01: ingest, collector, fabric-adapter;
- ULC-02: ingest, verifier, fabric-adapter;
- ULC-03: collector, verifier.

PASS means: only the intended roles are running and none is restart-looping.

Step 2 — Check health and readiness endpoints

The evidence containers expose status on container port 8080. Collector, verifier, and Fabric adapter bind host loopback; ingest binds the selected private host IP.

Use Docker's published-port mapping instead of guessing the host port:

```
for svc in ingest collector verifier fabric-adapter; do
  id="$(sudo docker ps -q --filter 'label=com.docker.compose.project=lorawan-gateway-evidence' --
filter "label=com.docker.compose.service=$svc" | head -n1)"
  [ -n "$id" ] || continue

  echo "=== $svc ==="
  endpoint="$(sudo docker port "$id" 8080/tcp | head -n1)"
  echo "$endpoint"
  host="${endpoint%:*}"
  port="${endpoint##*:}"
  [ "$host" = "0.0.0.0" ] && host=127.0.0.1

  curl --connect-timeout 3 --max-time 8 -fsS "http://${host}:${port}/readyz"
  echo
done
```

PASS means:

- ingest is ready when its PostgreSQL/object-store dependencies are usable;
- each collector is ready only when both broker sessions plus PostgreSQL/object storage are usable;
- each verifier is ready only when PostgreSQL, object storage, worker state, and trusted-decoder self-test are good;
- ULC-01 Fabric adapter reports enabled/ready;
- ULC-02 Fabric adapter reports healthy standby with enabled=false.

A /healthz response proves a process is alive; /readyz is the stronger dependency check.

Step 3 — Check recent service logs without dumping secrets

```
for svc in ingest collector verifier fabric-adapter; do
  id="$(sudo docker ps -q --filter 'label=com.docker.compose.project=lorawan-gateway-evidence' --
filter "label=com.docker.compose.service=$svc" | head -n1)"
  [ -n "$id" ] || continue
  echo "=== $svc ==="
  sudo docker logs --since=15m --tail=120 "$id"
done
```

Look for repeated DB/S3/TLS/MQTT errors, lease failures, trusted-decoder self-test failure, conflicting object identity, evidence integrity errors, or adapter reconciliation errors.

Do not enable debug logging if it would expose DSNs, authorization headers, payload secrets, or key material.

Step 4 — Verify the collector boundary

The commissioned collector contract is:

```
brokers:      10.104.0.2:8884 and 10.104.0.4:8884
TLS name:     mqtt.internal.lorawan.com
auth:         mTLS
topic:        as923/gateway+/event/#
```

```
permission:  READ only
publish:     denied
```

Each collector replica owns two distinct persistent client IDs, one per broker backend. The brokers do not replicate session state, so routing a collector only through the preferred/backup frontend is not equivalent.

If collector /readyz fails while the process lives, troubleshoot broker-1 session, broker-2 session, PostgreSQL, and object storage separately.

Step 5 — Verify evidence database state

Using an approved read-only PostgreSQL session to lorawan_telemetry, inspect outcomes:

```
SELECT status, count(*)
FROM gateway_evidence.event_verification
GROUP BY status
ORDER BY status;

SELECT gateway_id, last_sequence, server_received_at
FROM gateway_evidence.checkpoints
ORDER BY server_received_at DESC
LIMIT 10;
```

Interpret states literally. verified, evidence_gap, integrity_failure, and still-pending work have different meanings.

Do not update verifier-owned rows manually to make a downstream test pass.

Step 6 — Verify one lineage only when a real event is expected

For one known recent event, prove the chain in this order:

```
application event
-> MQTT witness object
-> gateway journal record
-> closed segment + predecessor chain
-> checkpoint
-> trusted decode
-> stored telemetry comparison
-> verifier-owned terminal result
```

Do not use dashboard appearance as acceptance proof. The verifier must reopen retained objects and recompute the required identities/hashes.

Step 7 — Understand recovery behavior

A WAN outage or one evidence-service replica loss does not require deleting or reconstructing authority state.

gateway journal	continues locally
local Mosquitto queue	grows if WAN is down
uploader	resumes from durable receipts
ingest replicas	accept exact retries idempotently
collector duplicates	collapse through deterministic capture identity
verifier leases	expire/reclaim after worker loss
telemetry	may continue while evidence stays pending
v2 Fabric eligibility	remains blocked until verifier-owned verified state exists

Availability-first telemetry must not be misrepresented as verified evidence.

Safe deployment/change procedure

1. Prove PostgreSQL, object storage, and cloud MQTT dependencies first.
2. Run the tracked host/staged deployment preflight appropriate to the change.
3. Keep image references immutable.
4. Change only the affected replica/profile.
5. Preserve separate workload identities.
6. Validate Compose with `docker compose config --quiet`.
7. Recreate/restart only the affected service.
8. Require `/readyz` after the change.
9. Verify one representative real lineage only when the change affects evidence semantics.
10. Keep Fabric activation as its own governed gate.

Do not bring every evidence replica and both Fabric adapters up blindly as one recovery action.

Gateway-side evidence rule

Gateway journal state is durable and is not normally reset. A deliberate fresh research epoch must quiesce the writer/uploader first and explicitly establish a new GENESIS boundary. Raw SeaweedFS history may remain retained even when active database metadata is intentionally reset.

Never delete current gateway journal/receipt state as an ordinary troubleshooting step.

Troubleshooting map

Symptom	First check
ingest not ready	PostgreSQL + SeaweedFS :18443 + TLS/mTLS
collector alive but not ready	both fixed broker sessions, then DB/S3
verifier alive but not ready	DB/S3 + trusted-decoder self-test + worker state
telemetry exists but verification remains pending	missing evidence/correlation, not Fabric
integrity_failure	preserve artifacts; investigate exact mismatch before retry
evidence_gap	missing lineage/checkpoint/MQTT witness; do not fabricate
duplicate collector observations	deterministic capture identity + DB uniqueness
one replica down	repair that replica; do not remove the HA responsibility
Fabric pending with attempts=0	check finalized payload + verifier eligibility first

Completion checklist

- intended two-replica ingest/collector/verifier placement is present;
- all active evidence services return `/readyz`;

- both collector replicas maintain the two fixed broker sessions;
- verifier trusted-decoder self-test passes;
- PostgreSQL evidence metadata is readable;
- SeaweedFS dependency is healthy;
- no immediate evidence_gap / integrity_failure condition exists for the event under test;
- ULC-01 adapter is the only enabled writer;
- ULC-02 adapter remains standby until its separate HA gate is accepted.

Detailed references

- [../../../../evidence-services/README.md](#)
- [../../../../evidence-services/cloud/deploy/README.md](#)
- [../server/integrations/gateway-integrity/04-service-architecture-and-runtime-contract.md](#)
- [../server/integrations/gateway-integrity/06-replicated-ha-deployment-journey.md](#)

Supporting runbook: README.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Gateway Security Evidence Services

This directory contains the implementation artifacts for the gateway-integrity v2 security-evidence path documented under `deployment/server/integrations/gateway-integrity/`.

Current implementation boundary

Implemented as repository source/static-policy artifacts:

- language-independent contracts for `gateway-journal-v1`, `gateway-journal-segment-v1`, `concentrator-d-uplink-correlation-v1`, `evidence-ingest-api-v1`, `evidence-ingest-receipt-v1`, `mqtt-capture-v1`, and `trusted-decoder-normalized-v1`;
- a dependency-light Go cloud module foundation;
- validated environment configuration with secret-safe summaries;
- structured JSON logging helpers;
- database capability interfaces shared by later repositories/workers;
- an immutable filesystem object-store backend for **development/smoke use only**;
- deterministic `mqtt-capture-v1` capture-key construction and fixed vector;
- independent Agriculture Kit payload-v2 trusted decoder source with fixed raw/normalized SHA-256 vector;
- evidence-ingest HTTP handler/core for checkpoints and closed segments, including stable evidence-ingest-receipt-v1 acknowledgements derived from persisted server acceptance time;
- direct verified-client-certificate identity binding for one 16-hex Gateway EUI;
- request-body/content-type/unknown-field/trailing-JSON controls;
- checkpoint semantic digest, duplicate/conflict handling, and monotonic-regression rejection contract;
- raw-segment object SHA verification and raw-object-before-metadata persistence ordering;
- memory metadata repository for unit/local smoke tests only;
- production PostgreSQL ingest repository using per-Gateway-EUI advisory transaction locks for atomic checkpoint/segment decisions;
- pgx-backed connection policy pinned to Go 1.25 / pgx v5.10.0 with hostname-verified TLS, SCRAM-only authentication, expected-role membership, and per-session writable-primary checks;
- executable `gateway-evidence-ingest` process wiring with graceful shutdown and direct mTLS listener construction;
- frozen `mqtt-collector-runtime-v1` dual-broker contract and Paho MQTT v5 source pinned at `github.com/eclipse/paho.golang v0.23.0`;
- `gateway-mqtt-evidence-collector` source with two independent persistent sessions to the physical Mosquitto backends, exact opaque payload capture, read-only subscription behavior, raw-object-before-metadata-before-ACK ordering, PostgreSQL duplicate convergence, and `/healthz/readyz/metrics` endpoints; event/up additionally receives the pinned `concentrator-d-uplink-correlation-v1` semantic projection only after raw storage succeeds;

- frozen verifier-runtime-v1 source boundary with v2 outbox discovery, SKIP LOCKED lease claims, lease-owner fencing, deterministic application/trusted-decoder comparison, exact first-reception provenance, raw MQTT object reopen/redecode, deterministic MQTT lookup, exact closed-journal parsing, complete predecessor-object chain verification through the matched segment, accepted-checkpoint digest recomputation, bounded retry, health/object-store readiness, full lineage persistence, and lease-fenced complete-lineage promotion to verified;
- trusted-decoder startup self-test plus build-time package-digest gate;
- gateway-evidence-verifier executable source using the gateway_evidence_verifier PostgreSQL authority boundary;
- separate gateway-fabric-adapter source with pinned Fabric Gateway v1.12.0/JCS v1.0.1, frozen v1/v2 startup vectors, immutable OpenBao seal persistence/verification, HRC Task 37 CreateSourceBoundAnchor submission using five positional metadata arguments plus the exact accepted payload bytes only as transient hrc.exact_payload, QuerySourceBoundAnchor / VerifySourceBoundDigest reconciliation, read-only reconstruction mode, and fail-closed pre-handoff standby runtime;
- a Rust 1.82 gateway runtime with separate writer/uploader executables, exact gateway-journal-v1 RFC 8785 hashing, canonical JSONL segments, monotonic crash-safe fsync-backed persistence/recovery, optional pinned ZeroMQ Concentrator subscriber, deterministic correlation, durable canonical receipt storage, HTTPS/mTLS curl transport, bounded retry/backoff, --sync-once, and continuous loops. The default source gate passes 28 tests plus format/Clippy/locked build; the OpenWrt package/runtime has since been commissioned on Gateway-01 and physically accepted with real RF/evidence lineage, while the tracked source/build material remains the reproducible rebuild path;
- migrations/001_gateway_evidence.sql with the evidence schema, worker-lease state, canonical lowercase Gateway-EUI constraints, database-level checkpoint monotonicity trigger, segment-1 GENESIS/late-predecessor constraints, verified-row projection completeness constraints, indexes, views, and least-privilege role grants;
- migrations/001_gateway_evidence.verify.sql for post-migration schema/ACL/invariant verification.

Current live/pending boundary:

- **cloud evidence runtime is commissioned / PASS:** SeaweedFS 4.41 S0-S9; the gateway_evidence PostgreSQL migration/HBA/CONNECT/six SCRAM LOGINS; the ten-role PgBouncer userlist on all three nodes; immutable GHCR image digests; Evidence PKI; four distinct read-only collector mTLS identities/ACLs; ingest-1/2; collector-1/2 each connected to both physical brokers; verifier-1/2 with the pinned trusted decoder; shared anchor :443 TCP/SNI routing; and Grafana evidence checkpoint/verification panels;
- Fabric Adapter-1 on ULC-01 is commissioned as the continuous HRC writer with FABRIC_ADAPTER_ENABLED=true and confirmed ledger transactions; Adapter-2 on ULC-02 remains immutable FABRIC_ADAPTER_ENABLED=false standby pending the HA fencing gate;
- the Rust writer/uploader source runtime and OpenWrt UCI/procd package are implemented and tested; Gateway-01 has passed target/runtime installation plus real IPC/RF/evidence acceptance. **Local evidence retirement/delete remains intentionally absent;**
- real commissioned-gateway journal/uploader + MQTT/application lineage has been accepted end-to-end, including verifier-owned verified rows from Gateway-01 / EMU-01 traffic; future RF tests should reuse this commissioned evidence lane rather than treating it as pending;
- the public ChirpStack/Evidence/MQTT normal path is commissioned on Reserved IPv4 129.212.208.168; only Reserved-IP reassignment/failover authority and controlled acceptance remain provider-dependent;
- Fabric ledger execution is active on ULC-01 through the governed HRC private Gateway; ULC-02 remains write-disabled until the HA lease-renewal/ownership-fencing gate is proven.

The repository host does not require a global Go installation. cloud/scripts/dev-build.ps1 bootstraps checksum-pinned Go 1.25.0 into project-local ignored paths, isolates all Go caches, and the current four-service tree passes gofmt, go test ./..., go build ./..., and Linux/amd64 builds. The four accepted cloud images are now

published and pinned by immutable GHCR digests in the production release file. The Rust side is pinned by gateway/rust-toolchain.toml to Rust 1.82.0; the current default writer/uploader source gate passes 28 tests total, formatting, Clippy, and locked build. The separate native/OpenWrt target build remains required for concentrator-dzmq and final gateway binaries.

Layout

```
evidence-services/  
  contracts/  
    evidence-ingest-api-v1/  
    evidence-ingest-receipt-v1/  
    gateway-journal-v1/  
    gateway-journal-segment-v1/  
    mqtt-capture-v1/  
    mqtt-collector-runtime-v1/  
    trusted-decoder-normalized-v1/  
    verifier-runtime-v1/  
  gateway/  
    Cargo.toml  
    Cargo.lock  
    src/  
    tests/  
  cloud/  
    go.mod  
    internal/  
      config/  
      database/  
      ingest/  
      logging/  
      mqttcapture/  
      mqttcollector/  
      objectstore/  
      uplinkcorrelation/  
      trusteddecoder/  
      verifier/  
    cmd/  
      evidence-ingest/  
      evidence-mqtt-collector/  
      evidence-verifier/  
  migrations/
```

The Go cmd/ trees contain source wiring; generated binaries stay in ignored .dev-out/. The Rust gateway tree contains the writer/uploader runtime plus the OpenWrt package source used by the physically accepted Gateway-01 deployment; BUILD.md is the reproducible rebuild path rather than an outstanding acceptance gate. Server reconstruction is defined in cloud/deploy/README.md and uses the same tracked Compose bundle on all three hosts with profile-specific host env files. Do not commit generated toolchains, caches, targets, binaries, live env files, credentials, or private keys.

Safety boundary

The filesystem object store and in-memory metadata repository are intentionally development/test-only. They prove API semantics such as create-if-absent, exact duplicate idempotency, conflicting duplicate rejection, checkpoint regression rejection, and SHA-256 verification. They do **not** satisfy the live cloud requirements for one-Droplet-loss raw-byte durability or PostgreSQL-backed metadata HA.

001_gateway_evidence.sql is the reproducible source of the already-commissioned live schema/ACL boundary. Do not reapply it to the current Patroni cluster; use it for rebuild/recovery or a deliberate forward migration.

Supporting runbook: README.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Reproducible Cloud Evidence Deployment Bundle

This directory is the tracked and **commissioned** server-side deployment bundle for the four Go evidence/attestation services: ingest, MQTT collector, verifier, and Fabric adapter. The base profile is live on the three cloud hosts. After the governed ULC-01 qualification, ULC-01 now runs the explicit enabled overlay as the continuous Fabric writer; ULC-02 remains intentionally disabled standby until the HA fencing gate passes. Rebuilding a server should require the repository plus protected credentials/PKI and the selected durable S3-compatible service; it should not require reconstructing Compose commands from chat history.

The Fabric adapter has two deliberately separate deployment states. The base `compose.yml` runs `adapter-1/2` with `FABRIC_ADAPTER_ENABLED=false`, so they expose only local health/readiness and do not open PostgreSQL, read an OpenBao SecretID, or contact Fabric. Actual ledger submission requires the explicit `compose.fabric-adapter-enabled.yml` overlay and must pass `fabric-adapter-enable-preflight.sh` first.

Frozen HA placement

```
ulc-01 / 10.104.0.2
  ingest-1
  collector-1
  fabric-adapter-1 enabled continuous writer -> HRC Fabric through the governed private Gateway

ulc-02 / 10.104.0.4
  ingest-2
  verifier-1
  fabric-adapter-2 disabled standby -> do not enable before HA fencing acceptance

ulc-03 / 10.104.0.8
  collector-2
  verifier-2
  no Fabric adapter worker
```

Every runtime starts with the documented ceiling 192 MiB / 0.20 CPU. That is a measurement guardrail, not permission to remove HA if the 2-GiB hosts are too small.

Files

<code>compose.yml</code>	base four-service definition; adapter is hard-frozen disabled
<code>compose.collector-mtls.yml</code>	collector client-certificate mounts
<code>compose.fabric-adapter-enabled.yml</code>	explicit activation overlay; never use for standby staging
<code>release.env.example</code>	template for immutable OCI digest references
<code>release.commissioned.env</code>	exact non-secret commissioned four-image GHCR release
<code>hosts/*.env.example</code>	exact node placement + host ports + enabled-adapter mount paths
<code>env/common.env.example</code>	non-secret common DB/S3 settings
<code>env/ingest.env.example</code>	protected ingest DB/S3 credentials + TLS paths

```

env/collector-password.env.example    legacy/reference input only; production preflight rejects password mode
env/collector-mtls.env.example        protected collector DB/S3 + client-certificate paths
env/verifier.env.example              protected verifier DB/S3 settings
env/fabric-adapter.env.example        protected DB/Fabric topology settings used only after handoff
host-preflight.sh                     early read-only live host/capacity/service/port gate
preflight.sh                          final base/standby staged-config gate
fabric-adapter-enable-preflight.sh    separate no-mutation gate before enabling ledger submission

```

The Compose definition runs every container as numeric 65532:65532, read-only root filesystem, all Linux capabilities dropped, no-new-privileges, PID limit 128, bounded Docker logs, and no in-image shell/health utility. Collector/verifier/Fabric-adapter health listeners bind host loopback only. Ingest binds only the selected private host IP/backend port. The public Evidence route is now commissioned through the shared HAProxy SNI frontend at smartagri-evidence.duckdns.org:443.

Inputs that intentionally remain external

A real base deployment must supply:

1. four OCI image **digest** references built from ../packaging/;
2. the selected SeaweedFS 4.41 raw-evidence cluster from seaweedfs/, commissioned with 010 cross-rack placement and the local internal-TLS create-only S3 frontend;
3. per-role PostgreSQL login credentials, each a member only of its corresponding NOLOGIN group role;
4. per-role S3 credentials with minimum object permissions;
5. Evidence server/client PKI and the commissioned PostgreSQL/MQTT/S3 CA files;
6. dedicated read-only MQTT collector broker identities;
7. free private/loopback host ports confirmed on the live server.

Fabric activation uses the commissioned HRC Task 37 handoff. The adapter contract is CreateSourceBoundAnchor(SourceRecordID, sourceType, producer, producedAt, schemaVersion) with the exact immutable finalized_payload bytes supplied only as transient hrc.exact_payload, followed by QuerySourceBoundAnchor(SourceRecordID) and VerifySourceBoundDigest(SourceRecordID, observedDigest). The frozen LoRaWAN source identity is FABRIC_AUTHENTICATED_SOURCE_SYSTEM_ID=lorawan-gateway-evidence; the private Gateway is 10.104.0.7:7051 with TLS name peer1.hrc.local, and ULC-01 has the commissioned dedicated non-admin adapter identity. ULC-02 remains a protected write-disabled standby until its HA fencing/ownership gate is accepted. The old 10.43.25.198:7051 K3s ClusterIP and legacy CreateAnchor, QueryAnchor, QueryAnchorByRecordID, and VerifyDigest APIs must not be used.

The repository does not guess image digests, secrets, or unused evidence-service host ports. The raw-store endpoint itself is now frozen to https://evidence-objects.internal.lorawan.com:18443; each evidence container resolves that name to its local host VPC IP, while SeaweedFS's raw S3 listener remains loopback-only.

Fixed host layout

Use these directories on every server:

```

/etc/lorawan-cloud/gateway-evidence/
  release.env
  host.env
  common.env
  ingest.env          # only on ingest hosts
  collector.env        # only on collector hosts
  verifier.env         # only on verifier hosts
  fabric-adapter.env   # ulc-01 active writer / ulc-02 protected disabled standby

```

```

/etc/lorawan-pki/gateway-evidence/
  postgres-ca.crt
  s3-ca.crt
  ingest-server.crt / ingest-server.key / gateway-client-ca.crt # ingest hosts
  mqtt-ca.crt                                           # collector hosts
  collector-broker*.crt/key                             # mTLS collector mode only

# ulc-01/02 only, created only at Fabric activation time:
/etc/lorawan-cloud/fabric-adapter/
  role_id
  secret_id

/etc/lorawan-pki/fabric/
  tls-ca.crt
  client.crt
  client.key

```

Protected role env files must be `root:root 0600`. Runtime private keys are bind-mounted into a numeric non-root container; install them as `root:<gid 65532> 0440`. Public certificates/CA files must also be readable by runtime `65532:65532`: use either `root:65532 0440` or a root-owned non-writable world-readable public file such as `0444`. The preflight checks actual mode/GID readability so a `0600 root:root` CA cannot pass and then break container startup.

If GID 65532 is unused on the server, a deployment operator may create a named host group for clarity before installing keys:

```
sudo groupadd --system --gid 65532 evidence-runtime
```

Do not change an existing GID 65532 owner blindly. If that numeric GID is already allocated, stop and review the container/runtime ownership design instead of weakening key permissions.

Prepare configuration

Copy examples, then replace placeholders **outside Git**:

```

sudo install -d -m 0750 -o root -g root /etc/lorawan-cloud/gateway-evidence
sudo install -d -m 0750 -o root -g root /etc/lorawan-pki/gateway-evidence

# Example only: choose the matching host file.
sudo install -m 0644 -o root -g root release.env /etc/lorawan-cloud/gateway-evidence/release.env
sudo install -m 0644 -o root -g root host.env /etc/lorawan-cloud/gateway-evidence/host.env
sudo install -m 0644 -o root -g root common.env /etc/lorawan-cloud/gateway-evidence/common.env
sudo install -m 0600 -o root -g root ingest.env /etc/lorawan-cloud/gateway-evidence/ingest.env

```

Do not copy literal example placeholders into production.

Database credentials

`001_gateway_evidence.sql` creates three NOLOGIN group roles:

```

gateway_evidence_ingestor
gateway_evidence_collector
gateway_evidence_verifier

```

The production LOGIN identities are now frozen and live. Preserve these mappings rather than creating new ad-hoc names:

```

evidence_ingest_ulc01    -> gateway_evidence_ingestor
evidence_ingest_ulc02    -> gateway_evidence_ingestor

```

```
evidence_collector_ulc01 -> gateway_evidence_collector
evidence_collector_ulc03 -> gateway_evidence_collector
evidence_verifier_ulc02  -> gateway_evidence_verifier
evidence_verifier_ulc03  -> gateway_evidence_verifier
```

All six are SCRAM LOGIN roles with exactly one intended NOLOGIN authority membership and inherited CONNECT on `lorawan_telemetry`. Protected plaintext custody remains outside Git/Markdown under the commissioned root-only bootstrap boundary.

Every DSN uses the commissioned logical endpoint `pgbouncer.internal.lorawan.com:6432`, database `lorawan_telemetry`, hostname-verified TLS, and the mounted PostgreSQL CA. Because this internal name is not assumed to exist in public DNS, Compose maps it inside every container to the **local node private IP** (`10.104.0.2`, `.4`, or `.8`), preserving the local PgBouncer -> local HAProxy -> Patroni-primary path. The Go pool also enforces SCRAM-SHA-256 and verifies role membership/writable-primary routing on every new physical DB session.

All six PostgreSQL LOGIN roles and the three-node PgBouncer expansion are commissioned. Every PgBouncer node has the exact ten-entry/six-evidence SCRAM userlist SHA-256 `665f1592c96ca276681a454b9cbcd6ab8ab0cbfb4594b8ddd443239db58df391`, with the original four-role verifier set preserved for rollback and proven byte-identical across the expansion. Fresh strict verify-full evidence authentication passed through all three physical `:6432` endpoints and routed to the current writable Patroni primary. Do not regenerate PgBouncer userlists again unless a later credential change requires it.

S3 credentials

The production object-store backend is selected and commissioned through S9: SeaweedFS 4.41 behind the internal HAProxy TLS/create-only endpoint `https://evidence-objects.internal.lorawan.com:18443`. The locked production helper write and cross-host retained-object verification are PASS. Runtime S3 identities are already installed with the following permission split:

```
ingest:    HeadBucket + GetObject + PutObject on evidence prefix
collector: HeadBucket + GetObject + PutObject on evidence prefix
verifier:  HeadBucket + GetObject only on evidence prefix
```

The application uses conditional create-if-absent writes. Do not grant object delete/overwrite administration to these runtime credentials.

Collector broker authentication

Production collector authentication is frozen to **mTLS** because the current commissioned `ulc-01:8884` and `ulc-02:8884` gateway-facing broker listeners require client certificates and map certificate identity to the MQTT username. Do not reopen password mode merely for convenience.

```
collector-mtls.env.example -> /etc/lorawan-cloud/gateway-evidence/collector.env
EVIDENCE_COLLECTOR_AUTH_MODE=mtls
```

Issue dedicated collector `clientAuth` identities whose Mosquitto usernames receive only `read` `as923/gateway/+/#/event/#`; grant no publish or command permission. `compose.collector-mtls.yml` is supplied automatically by `preflight.sh` and must also be supplied to the later activation command. The password example remains reference material only and is rejected by production `preflight`.

Each collector replica has **two distinct client IDs**, one per fixed backend (`10.104.0.2:8884` and `10.104.0.4:8884`). Across two collectors that produces four independent persistent read-only sessions.

Fast live-host preflight

Before creating evidence credentials, applying the migration, or choosing final ports, run the same observation-only check from an **already-authorized SSH session** on each server:

```
sudo bash host-preflight.sh
```

Run it on ulc-01, then ulc-02, then ulc-03. It verifies current private-IP identity, memory/disk headroom, UID/GID 65532 collision risk, Docker/Compose/Buildx availability, container RSS, HAProxy/PgBouncer state, Patroni/etcd visibility, current :8884 mTLS policy where Mosquitto lives, the existing anchor :443, and dynamically suggests currently free ingest/health ports. Suggested ports are observations only; they are not reserved and should be copied into protected host configuration immediately before the final staged-config preflight.

The script deliberately does not require the repository runner to possess server private keys. Never copy a workstation SSH private key into this repository or onto a Droplet merely to automate this check.

Mandatory final staged-config preflight

After files are installed, but **before pulling or starting anything**:

```
sudo ./preflight.sh \  
  /etc/lorawan-cloud/gateway-evidence/release.env \  
  /etc/lorawan-cloud/gateway-evidence/host.env
```

It verifies:

- exact frozen node/profile placement, including adapter-1 on ulc-01 and adapter-2 on ulc-02 only;
- correct private IP on the host;
- all four image references are immutable @sha256 references;
- the base Fabric adapter profile remains standby-only and accepts no DB/OpenBao/Fabric credential mounts;
- selected host ports are numeric and currently free;
- protected env/key ownership and permissions;
- required CA/certificate files are actually readable by runtime 65532:65532 and private keys are root:65532 0440;
- pgbouncer.internal.lorawan.com is mapped inside containers to the local node private IP;
- production backend is s3, never the dev filesystem;
- collector auth-mode-specific inputs;
- PostgreSQL DSNs use the logical PgBouncer endpoint + verify-full;
- docker compose config --quiet succeeds.

Expected final marker:

```
EVIDENCE_DEPLOYMENT_PREFLIGHT=PASS
```

The script does not pull images, start containers, issue credentials, alter permissions, or mutate the database.

Base/standby activation command

This boundary was commissioned on 2026-09-01 after image digests, SeaweedFS S9, the DB/PgBouncer roles, Evidence PKI, shared-443 design, and all three host preflights passed. The command remains the reproducible start/recovery path. Production collectors use mTLS, so collector hosts include the collector override:

```
sudo docker compose \  
  --env-file /etc/lorawan-cloud/gateway-evidence/release.env \  
  --env-file /etc/lorawan-cloud/gateway-evidence/host.env \  
  -f compose.yml \  
  -f compose.collector-mtls.yml \  
  up -d
```

On ulc-02 there is no collector profile, so use only `compose.yml`. The Fabric adapter profile is present on ulc-01/02 but remains disabled by the base Compose definition. Start and verify one logical boundary at a time; do not bring all six evidence replicas plus two adapter standbys up blindly.

Fabric adapter activation is a separate gate

Do not issue/install an OpenBao SecretID merely because the standby container is healthy. After the external Fabric handoff is complete, install the protected adapter env/identity files, then run:

```
sudo ./fabric-adapter-enable-preflight.sh \  
  /etc/lorawan-cloud/gateway-evidence/release.env \  
  /etc/lorawan-cloud/gateway-evidence/host.env
```

This preflight verifies the exact adapter host/worker identity, protected file modes, `fabric_adapter` PgBouncer TLS DSN, fixed OpenBao service path, Fabric certificate/private-key match, stable OpenBao TLS/health, external Fabric Gateway server TLS, and the combined Compose configuration. It performs no pull/start/credential issuance or state mutation.

Only after that marker is `FABRIC_ADAPTER_ENABLE_PREFLIGHT=PASS` may the explicit overlay be used for the one adapter being activated:

```
sudo docker compose \  
  --env-file /etc/lorawan-cloud/gateway-evidence/release.env \  
  --env-file /etc/lorawan-cloud/gateway-evidence/host.env \  
  -f compose.yml \  
  -f compose.fabric-adapter-enabled.yml \  
  up -d fabric-adapter
```

On ulc-01 include `-f compose.collector-mtls.yml` as well when managing the whole host stack. If the external Fabric Gateway requires a separate **transport mTLS** client certificate rather than server-auth TLS plus the Fabric application identity currently implemented, stop at the preflight failure and extend/rebuild the adapter; never disable TLS verification to force the connection.

Commissioned immutable release - 2026-09-01

Tracked `release.commissioned.env` and the production release files use these immutable references. The current hosts may use protected exact preloaded-image overrides with `--pull never` because the legacy host registry credential cannot read the new private GHCR packages; do not weaken package privacy as a workaround. A rebuild should either provision authorized registry-read access or preload images whose content identity has been verified against the commissioned release.

```
EVIDENCE_INGEST_IMAGE=ghcr.io/jervis-umtc/lorawan/gateway-evidence-
ingest@sha256:f2fd695e8d2505a12202b4fabd92d035c562e16d4e308ccf9436a08a36aaec1a
EVIDENCE_COLLECTOR_IMAGE=ghcr.io/jervis-umtc/lorawan/gateway-mqtt-evidence-
collector@sha256:0bcaaa127f913157596a87077235d4f0717fa1c29f08d8aca8568c3167ed2cff
EVIDENCE_VERIFIER_IMAGE=ghcr.io/jervis-umtc/lorawan/gateway-evidence-
verifier@sha256:3416496fa563154fbd7209a575acd040b735b3c4f4159902fc886e78167195c0
EVIDENCE_ADAPTER_IMAGE=ghcr.io/jervis-umtc/lorawan/gateway-fabric-
adapter@sha256:cd4308e8985d74ea7fab957a2a4adadd1831b776a8f9af2fdd6291179df83e7a
```

All three authoritative base `preflight.sh` runs passed. Current placement is `ulc-01=ingest-1+collector-1+adapter-1-enabled`, `ulc-02=ingest-2+verifier-1+adapter-2-disabled`, `ulc-03=collector-2+verifier-2`. The base Compose remains deliberately fail-closed; ULC-01 reaches the enabled state only through the governed activation overlay and `preflight`. Every runtime uses numeric `65532:65532`, read-only rootfs, `cap_drop: ALL`, `no-new-privileges`, `pids_limit: 128`, `192 MiB`, `0.20 CPU`, bounded logs, and `restart: unless-stopped`.

SeaweedFS S9, Evidence PKI, four distinct read-only collector mTLS identities/ACLs, the dual-broker collector readiness gate, replicated ingest/verifier readiness, and the private/shared-443 normal path are PASS. ULC-01 Adapter-1 is the governed continuous Fabric writer with `FABRIC_ADAPTER_ENABLED=true`; ULC-02 Adapter-2 remains healthy `FABRIC_ADAPTER_ENABLED=false` standby until the HA fencing gate passes.

What is still not claimed

The real Gateway-01 / assembled EMU-01 v2 physical evidence lineage is PASS through Concentrator, MQTT, journal/uploader, Evidence ingest, checkpoints/segments, and trusted-decoder verification. On the final 2026-09-02 shutdown check, Gateway-01 had closed and receipted segment 28, its open journal was empty, and the cloud held 492 verified v2 events with 0 pending, 0 unseen, and 0 `evidence_gap / integrity_failure` rows. Both verifier replicas run the corrected continuous-discovery build that excludes already-seen v2 keys before applying the 100-row batch limit, preventing the old first-100 starvation condition. Public ChirpStack/Evidence/MQTT normal paths are also PASS. Remaining external claims are deliberately narrower: Reserved-IP reassignment/failover authority and controlled acceptance, plus the separate ULC-02 Fabric HA fencing/dual-worker acceptance gate. Do not redo the already-passed cloud evidence commissioning without a relevant state change.

Supporting runbook: 04-service-architecture-and-runtime-contract.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

4. Canonical Integrity-Watcher Service Architecture

This document is the canonical runtime architecture for the software-only gateway integrity and Fabric-attestation path.

Use it to answer one question consistently across the project:

Which long-running service observes each boundary, what durable state does it own, and what is allowed to happen before an event can reach OpenBao and Hyperledger Fabric?

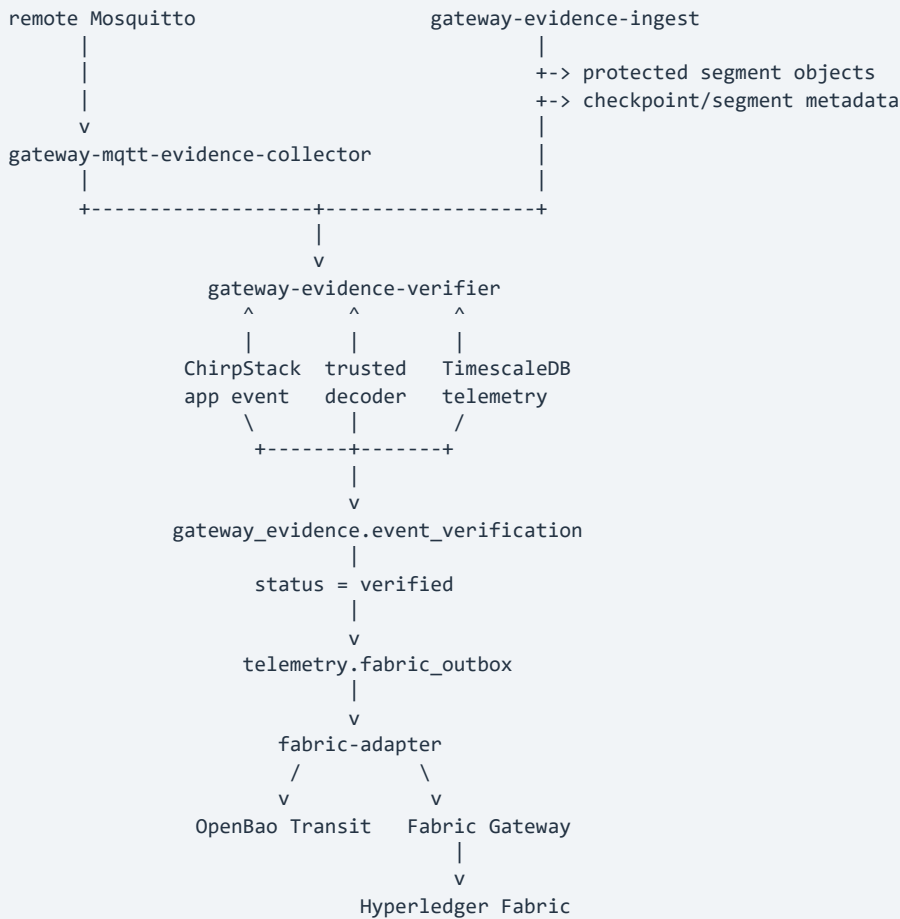
The design deliberately does **not** use one privileged process that “watches everything.” The system is split into small services with different credentials and write permissions. PostgreSQL and protected evidence storage carry durable state between them.

The individual data contracts remain authoritative in the gateway journal, server verifier, and Fabric-attestation manuals. This page defines how those contracts fit together as one deployable architecture.

4.1 Architectural rule: verification is a pipeline, not a mega-watcher

The safe design is:





There are two independent evidence branches before the verifier:

```

Gateway journal path
  Concentrator -> journal -> uploader -> evidence ingest

Remote delivery witness path
  Concentrator -> MQTT Forwarder -> Mosquitto -> cloud MQTT collector
  
```

They meet only in the verifier. The journal never feeds ChirpStack, and the MQTT collector never writes the gateway journal.

4.2 Service inventory and ownership

Service 1 - gateway-integrity-journal

Location: physical Raspberry Pi gateway.

Purpose: record what the supported Concentrator event interface exposed before application decoding and before the ordinary MQTT delivery path can transform the event.

Reads:

```
supported Concentrator event/IPC interface - read only
```

Writes:

```
journal open segment
journal sequence state
record hashes
segment hashes
```

Must not own:

```
RAK5146 SPI control
Mosquitto administration
cloud evidence database credentials
OpenBao credentials
Fabric identity
```

For each accepted source event it performs:

```
source event
-> versioned record_body
-> add gateway sequence + previous_record_hash
-> RFC 8785 canonicalization
-> UTF-8 bytes
-> SHA-256
-> append {record_body, record_hash}
-> advance sequence/hash state
```

The record chain answers:

```
Did a complete recorded gateway observation change, disappear from the middle, move, or
break continuity?
```

Service 2 - gateway-journal-uploader

Location: physical Raspberry Pi gateway.

Purpose: move already-created evidence off the gateway without making WAN availability a prerequisite for local evidence recording.

Reads:

```
closed journal segments
current checkpoint state
upload acknowledgement state
```

Writes locally:

```
upload receipt/acknowledgement state only
```

Network access:

```
HTTPS + mutual TLS -> gateway-evidence-ingest
```

The uploader does not rewrite record bodies or recompute historical content to make an upload pass. It sends closed segments and checkpoints and accepts only a response that identifies the expected gateway, segment/checkpoint identity, and digest.

A WAN outage therefore behaves as:

```
journal writer      continues
Mosquitto queue     grows
```

journal uploader closed segments	cannot advance cloud anchor remain local
-------------------------------------	---

When WAN returns, MQTT delivery and journal upload recover independently.

Service 3 - gateway-evidence-ingest

Location: protected cloud/server application tier.

Purpose: authenticate each gateway upload and append accepted evidence to server-side durable storage.

Responsibilities:

1. terminate/validate the approved HTTPS + mTLS identity boundary;
2. map one upload identity to exactly one Gateway EUI;
3. apply body-size and version limits;
4. calculate the uploaded object SHA-256;
5. reject conflicting retries;
6. store raw closed-segment objects without overwrite;
7. insert checkpoint/segment metadata and receipt information;
8. return a receipt that identifies the accepted object.

It does **not** declare an application event verified. Ingest means “accepted for verification,” not “trusted.”

Service 4 - gateway-mqtt-evidence-collector

Location: cloud/server application tier.

Purpose: independently preserve what the remote MQTT broker actually received from gateways.

Broker permission:

READ gateway event topics only WRITE denied command topics denied

For each matching broker event it preserves:

exact serialized event or protected object reference serialized_event_sha256 PHYPayload digest when available gateway ID broker receive time correlation identifiers radio metadata required by the versioned matching contract collector version
--

This service is an independent delivery witness. It does not replace ChirpStack and cannot publish corrective gateway events.

Service 5 - gateway-evidence-verifier

Location: cloud/server application tier.

Purpose: turn independent evidence into one durable verification result.

This is the correlation engine, but it is still not a Fabric signer.

Its work is split into two logical loops so initial work creation is explicit:

```
discovery/reconciliation loop
-> scan durable evidence-selected application work, primarily telemetry-attestation-v2 source/outbox
identities
-> idempotently ensure one gateway_evidence.event_verification row exists
-> new work begins as status=pending
-> duplicate discovery uses the unique source identity and does not create a second authority row

verification worker loop
-> claim pending work with DB lease and commit before evidence reads
-> load exactly one accepted application source + first-reception provenance
-> locate MQTT by exact Gateway-EUI/uplink-ID/frequency/context tuple
-> reopen raw MQTT object, recompute object/capture identity, re-decode pinned gw.UplinkFrame
-> recompute concentrator-uplink-correlation-v1 and compare application reception metadata
-> locate the journal record carrying that semantic digest
-> reopen and exactly verify the matching closed segment object
-> reopen and exactly verify every predecessor segment object back to segment 1 / GENESIS
-> verify record hashes, previous_record_hash links, segment hashes, and cross-segment continuity
-> load the accepted checkpoint for the matched segment final sequence and recompute checkpoint_digest
-> run the pinned trusted decoder on exact application bytes
-> compare the matching TimescaleDB normalized rows
-> persist the complete lineage projection and atomically promote the owned pending row to status=verified
```

Node-RED must not create verified state, and the Fabric adapter must not create verifier authority rows. The verifier is now the sole application-level author of that transition: CompleteVerified is lease-owner fenced, requires the row to remain pending, clears the worker lease, records verified_at, and writes the complete lineage projection in the same update. The migration additionally rejects incomplete verified rows. This does not claim a live verified event before the production evidence stack and real gateway lineage are commissioned. A separate Kafka/RabbitMQ/Valkey queue is not required for this low-rate design because durable discovery, uniqueness, retries, and leases already live in PostgreSQL.

The verifier owns transitions into states such as:

```
pending
verified
evidence_gap
integrity_failure
not_required
```

It must not possess:

```
OpenBao sign permission
OpenBao root/unseal material
Fabric client private key
Fabric submit authority
MQTT publish permission
```

Component 6 - pinned trusted decoder

The trusted decoder is a deterministic service or library used by the verifier. It is independent of Node-RED's operational normalization flow.

Input:

```
accepted raw ChirpStack application data
```

Output:

```
versioned deterministic normalized metric object
```

The verifier compares this result with the stored telemetry row. This prevents the Node-RED flow from becoming its own evidence authority.

The decoder version or code identity must be pinned and recorded with the verification result.

4.2A Replication and failover model

Replication is defined per responsibility; do not clone processes without defining how duplicate work becomes safe.

Gateway edge	
gateway-integrity-journal	one physical gateway; crash-safe local state, not fake HA
gateway-journal-uploader	one physical gateway; restart/resume from durable acknowledgement state
Cloud evidence tier	
gateway-evidence-ingest	2 replicas, active/active
gateway-mqtt-evidence-collector	2 replicas, active/active
gateway-evidence-verifier	2 replicas, active/active DB-leased workers
trusted decoder	same immutable/stateless package on both verifier replicas
raw evidence storage	cross-host durable; must survive one Droplet loss
gateway_evidence PostgreSQL	inherits commissioned 3-node Patroni HA
Later sealing tier	
OpenBao	commissioned 3-node Raft HA
fabric-adapter	target two lease-safe workers

Ingest replicas: both may receive the same gateway retry. Checkpoint (`gateway_id`, `last_sequence`) and segment (`gateway_id`, `segment_id`) identities are authoritative. Exact duplicate content is idempotent; different content under an existing identity is a security conflict. Raw objects are create-only. A response is not successful until both the raw-object durability rule and PostgreSQL metadata commit are satisfied.

Collector replicas: each replica uses its own revocable MQTT client identity and maintains persistent read-only sessions to both commissioned broker backends, not merely one session through the active/standby HAProxy route. The brokers do not replicate session state. Both collector replicas may therefore observe the same event. A versioned deterministic `capture_key_sha256` plus create-if-absent object semantics and a database uniqueness constraint collapse those duplicate observations into one logical MQTT counterpart.

Verifier replicas: both run the same reviewed image and trusted-decoder digest. Pending `event_verification` rows are claimed with a short `FOR UPDATE SKIP LOCKED` transaction plus expiring `worker_id` lease. The worker commits the lease before object reads/decoding and clears it only when persisting a terminal result. Expired leases are reclaimable after a crash. No verifier replica receives OpenBao or Fabric authority.

Raw evidence storage: two local directories are not replication. The selected live backend must provide at least two independent durable copies or equivalent erasure/replication semantics across failure domains. Initial commissioning proves the configured durability policy plus normal create/get/exact-SHA behavior; the deliberate one-Droplet-loss recovery exercise is deferred to Guide 3 / Phase 15. PostgreSQL `object_ref` remains logical/stable even if a storage replica moves. The commissioned Patroni replicas currently stream asynchronously, so PostgreSQL HA alone must not be treated as proof that a newly acknowledged raw evidence byte-object already exists on another member at ACK time. Keep PostgreSQL as metadata/state unless an explicit stronger raw-byte durability mechanism is separately proven.

Single-host loss behavior: normal telemetry may continue; evidence work may pause briefly but must resume without rewriting historical objects, creating a second authoritative MQTT counterpart, or losing an already-accepted checkpoint. A replica loss is a Phase 15 failure test, not part of commissioning.

Durable state - gateway_evidence

PostgreSQL carries evidence indexes, hashes, references, and verification state. Large raw objects remain in protected evidence/object storage.

Key objects are:

```
gateway_evidence.checkpoints
  server-side anchors of accepted gateway history

gateway_evidence.segments
  closed-segment metadata, hashes, object references, verification status

gateway_evidence.mqtt_gateway_events
  remote broker witness indexes/hashes

gateway_evidence.event_verification
  authoritative per-application-event verification result
```

The database is the durable handoff between the evidence services and the later Fabric worker. Services must not rely on an in-memory queue as the only copy of security state.

Durable state - telemetry.fabric_outbox

The outbox is the asynchronous boundary between telemetry ingestion and blockchain availability.

Node-RED may atomically enqueue an attestation job with accepted telemetry, but Node-RED does not verify gateway evidence, create the OpenBao signature, or submit to Fabric.

For telemetry-attestation-v2, an outbox row is eligible for sealing only when the matching verifier-owned row is exactly:

```
status = verified
```

Service 7 - fabric-adapter

Location: protected application host/service tier.

Purpose: consume eligible durable outbox work, create the canonical attestation, seal it with OpenBao, submit it through Fabric Gateway, and reconcile the final ledger result.

It watches only durable database work. It does **not** watch Concentrator, the journal directory, or MQTT gateway topics.

Its work loop is:

```
claim eligible outbox row with FOR UPDATE SKIP LOCKED
-> commit worker lease
-> load fixed source projection
-> for v2 load exactly one status=verified gateway_evidence result
-> require immutable finalized_payload exact JSON bytes
-> build/verify the separate local versioned evidence seal
-> RFC 8785 JCS + SHA-256 for the local OpenBao evidence boundary
-> request OpenBao Transit signature and persist/verify that seal
-> SHA-256 the exact finalized_payload bytes for HRC Task 37
-> prepare CreateSourceBoundAnchor with five metadata args
  + transient hrc.exact_payload = exact finalized_payload bytes
-> persist txid + record_id + endorsed transaction bytes + signed commit-status request
-> submit the already-prepared transaction through Fabric Gateway
-> recover/wait for commit status
```

- > QuerySourceBoundAnchor + VerifySourceBoundDigest
- > confirmed only after authoritative commit + exact source-bound verification

If final commit state is uncertain:

```
reconciling
-> recover the same durable prepared transaction/commit-status state
-> query/reconcile the source-bound ledger record
-> do not blindly generate a new transaction

needs_attention
-> permanent commit/anchor/digest/security conflict
-> operator review required
```

4.3 Trust and permission matrix

Component	Gateway journal write	MQTT gateway topics	Verification-state write	Telemetry/outbox write	OpenBao sign	Fabric submit
gateway-integrity-journal	Own evidence only	No	No	No	No	No
gateway-journal-uploader	No historical rewrite	No	No	No	No	No
gateway-evidence-ingest	Server append only	No	No	Evidence metadata only	No	No
gateway-mqtt-evidence-collector	No	Read only	No	Capture indexes only	No	No
gateway-evidence-verifier	No	No publish	Yes, verifier result only	Read telemetry	No	No
Node-RED	No	Application subscription only	No	Telemetry + enqueue only	No	No
fabric-adapter	No	No	Read only	Restricted outbox state/seal update	Yes, Transit policy only	Yes
Grafana	No	No	No	Read only	No	No

This separation is a security property. A component compromise should not automatically grant enough authority to fabricate the complete evidence chain and blockchain attestation.

4.4 Exact end-to-end lifecycle for one uplink

Stage 1 - Observe

```
RAK5146 -> Concentrator
```

Concentrator remains the only radio owner.

Stage 2 - Fan out before application processing

```
Concentrator -> MQTT Forwarder  
Concentrator -> gateway-integrity-journal
```

The two paths are independent.

Stage 3 - Journal locally

The journal assigns its own monotonic gateway sequence, links the previous record hash, canonicalizes the record, computes SHA-256, and appends the complete record.

Stage 4 - Deliver normally

MQTT Forwarder publishes to gateway-local Mosquitto. Mosquitto provides the bounded persistent delivery queue and bridges gateway events to the remote broker.

Stage 5 - Anchor evidence off-device

The uploader sends checkpoints during healthy connectivity and uploads closed segments. The server stores accepted anchors outside the Raspberry Pi.

Stage 6 - Capture an independent remote MQTT witness

The read-only MQTT collector stores what the cloud broker actually received before ChirpStack application processing.

Stage 7 - Process the application event

ChirpStack performs the normal LoRaWAN processing and publishes the accepted application event. Node-RED performs operational validation/normalization and persists telemetry.

Stage 8 - Verify source lineage

The verifier proves journal hash/segment/checkpoint continuity and correlates the journal observation with the remote MQTT witness and the accepted ChirpStack event.

Stage 9 - Independently decode and compare

The pinned trusted decoder reconstructs the approved normalized values from accepted raw application data. The verifier compares that deterministic result with TimescaleDB.

Stage 10 - Persist authoritative verification state

The verifier writes one `gateway_evidence.event_verification` row.

Only an unambiguous successful lineage becomes:

Stage 11 - Gate v2 work

The Fabric Adapter's claim query requires the matching v2 verification row to be verified.

pending, evidence_gap, and integrity_failure must never be silently promoted just to drain an outbox queue.

Stage 12 - Require exact finalized payload and build the local evidence seal

The Fabric Adapter can claim a row only when immutable finalized_payload exists; v2 also requires verifier-owned verified state. It preserves those exact payload bytes for HRC and separately builds/verifies the versioned local evidence object protected by OpenBao.

Stage 13 - Hash and seal

```
canonical evidence
-> RFC 8785
-> UTF-8
-> SHA-256
-> OpenBao Transit ECDSA P-256 signature
```

The journal record hash and the Fabric evidence digest are different hashes with different jobs:

```
journal hash
protects ordered gateway-history consistency

Fabric evidence digest
identifies the complete verified attestation object
```

Stage 14 - Persist the seal before network submission

The Adapter stores the canonical JSON, SHA-256 digest, signature algorithm, OpenBao key-version ID, complete versioned signature, and seal timestamp before the first Fabric network call.

Stage 15 - Prepare and submit the source-bound HRC transaction

The adapter calls CreateSourceBoundAnchor(SourceRecordID, sourceType, producer, producedAt, schemaVersion) and supplies the exact immutable finalized_payload bytes only as transient hrc.exact_payload. Before the first orderer submit it durably stores the Fabric transaction ID, HRC record ID, endorsed transaction bytes, and signed commit-status request. Full gateway forensic evidence remains off-chain.

Stage 16 - Confirm or reconcile

A row becomes confirmed only after successful commit status plus matching QuerySourceBoundAnchor(SourceRecordID) and VerifySourceBoundDigest(SourceRecordID, sha256(finalized_payload))=MATCH. Uncertain post-submit state becomes reconciling; permanent commit/anchor/digest conflicts become needs_attention. The migration still permits legacy submitted_unknown rows for compatibility, but the current worker does not emit that state.

4.5 Service state machines

Journal/uploader

```
RECORDING
|
+--> segment closes -> READY_TO_UPLOAD
|
|   +-> WAN unavailable -> remain local
|   |
|   +-> accepted receipt -> ACKNOWLEDGED
```

A complete invalid historical record does not transition back into normal recording through automatic repair. It raises an integrity/recovery fault.

Segment verification

```
pending
|
+--> all cryptographic/lineage checks pass -> verified
|
+--> required evidence unavailable -----> evidence_gap
|
+--> contradictory bytes/hash/history -----> integrity_failure
```

Per-event verification

```
pending
|
+--> unique full lineage + decoder match -> verified
|
+--> required counterpart never arrives -> evidence_gap
|
+--> payload/hash/normalized conflict ---> integrity_failure
```

Fabric outbox

```
pending / failed
|
v
processing --valid commit--> confirmed
|
|   +--transient pre-submit failure--> failed + backoff
|   |
|   +--uncertain post-submit result--> reconciling -> recover/query/commit-status reconcile
|   |
|   +--permanent commit/anchor/security conflict--> needs_attention
|   |
|   +--other exhausted/permanent local work--> dead_letter
```

For v2, pending outbox status alone does not mean eligible. The matching gateway verification must also be verified.

4.6 Startup and dependency order

Gateway startup

1. persistent gateway storage mounts
2. Concentrator starts and owns RAK5146
3. gateway-integrity-journal verifies/reopens local chain
4. local Mosquitto starts
5. MQTT Forwarder starts
6. network/backhaul becomes available
7. Mosquitto bridge reconnects
8. gateway-journal-uploader resumes checkpoints/closed segments

Evidence failure does not automatically stop telemetry delivery. The default architecture is availability-first: telemetry may continue while the evidence state becomes pending/gapped. That condition must remain visible and must block v2 promotion as required.

Cloud/server startup

A safe logical order is:

1. PostgreSQL/TimescaleDB and protected evidence storage available
2. cloud MQTT available
3. gateway-evidence-ingest starts
4. gateway-mqtt-evidence-collector starts
5. ChirpStack/application path available
6. gateway-evidence-verifier starts after DB/evidence dependencies are readable
7. OpenBao KMS available and unsealed
8. fabric-adapter starts only on the explicitly enabled owner after reviewed image, credentials, HRC handoff, and activation preflight pass

Services may restart independently because security state is durable. Restarting one evidence replica, the verifier pool, or the Fabric Adapter must not erase jobs or require reconstructing previously sealed evidence from mutable current telemetry. Replication never changes trust ownership: two ingestors are still only ingestors, two collectors are still read-only witnesses, and two verifiers still cannot sign.

4.7 WAN outage and recovery sequence

During gateway WAN loss:

LoRaWAN reception	continues
journal sequence/hash chain	continues
local journal segments	continue/rotate
Mosquitto persistent queue	grows
cloud MQTT witness	receives nothing new
cloud checkpoint	must not falsely advance
verification	remains pending for missing remote evidence

After WAN returns:

1. Mosquitto bridge reconnects and drains gateway events.
2. MQTT evidence collector captures those delivered events.
3. Journal uploader sends missing closed segments/checkpoints.
4. Evidence ingest stores them without replacing prior accepted evidence.
5. Verifier proves continuity from the latest accepted cloud anchor.
6. Verifier correlates journal records to MQTT captures.
7. Verifier links accepted ChirpStack application events.
8. Trusted decoder is compared with TimescaleDB.

9. Result becomes verified / evidence_gap / integrity_failure.
10. Only verified v2 events become eligible for OpenBao/Fabric work.

Delivery recovery and evidence recovery are separate. One becoming healthy does not imply the other is healthy.

4.8 What “monitoring the architecture” actually means

There is no additional privileged watcher required to decide whether the pipeline is healthy. Each service records durable state, and read-only operational views/Grafana can inspect that state.

Monitor at minimum:

Gateway

```
journal process/restarts
last journal sequence
last record hash
open/closed segment IDs
journal storage usage
unuploaded segment count/bytes
last accepted checkpoint receipt age
uploader retry/error state
Mosquitto queue usage and drain state
```

Evidence server

```
checkpoint ingest success/conflicts
segment upload backlog
oldest pending segment verification
gateway MQTT collector connectivity/lag
unmatched journal records
unmatched MQTT events
pending event-verification age
evidence_gap count
integrity_failure count
trusted decoder mismatch count
```

Fabric gate

```
v2 outbox rows blocked waiting for verification
verified-but-not-sealed age
failed adapter work
reconciling count/age
legacy submitted_unknown count/age (migration compatibility only)
needs_attention count/age
dead_letter count
OpenBao sign/verify failures
Fabric commit/reconciliation failures
```

A key diagnostic example is:

```
telemetry fresh + MQTT healthy + checkpoint stale
```

This means delivery is healthy while the evidence uploader/ingest path is degraded. Do not report the whole gateway as simply “healthy.”

4.9 Failure ownership

Failure	Normal telemetry	Verification result	Fabric v2 consequence
Journal service down	May continue	Pending -> gap under policy	Blocked
WAN down	Buffered locally	Pending while remote evidence absent	Blocked until recovery/verification
Journal hash conflict	May continue	integrity_failure	Permanently blocked/security path
Journal segment missing	May continue	evidence_gap if contradiction not proven	Blocked or explicit business gap policy
MQTT collector down	Delivery may still work	Pending -> gap if evidence expires	Blocked
ChirpStack event missing	No accepted app lineage	Pending/gap	Blocked
Trusted decoder mismatch	Telemetry exists	integrity_failure	Blocked
Evidence verifier down	Telemetry remains available	Stays pending	Blocked
OpenBao down	Telemetry + verified evidence remain durable	Remains verified	Outbox waits/fails with retry policy
Fabric unavailable	Telemetry + seal remain durable	Remains verified	Outbox retries/reconciles; ingestion unaffected

4.10 Implementation order

Build and commission in this order. Do not start at Fabric and work backward.

1. **Journal source adapter + fixed record test vector** — prove the exact supported Concentrator input and RFC 8785/SHA-256 output.
2. **Crash-safe journal/segment storage** — prove sequence continuity, reboot recovery, torn-tail handling, and segment chaining.
3. **Journal uploader + mTLS receipt contract** — prove checkpoints and closed-segment upload without coupling recording to WAN health.
4. **Evidence ingest + protected object store** — prove identity binding, idempotency, conflict rejection, and no overwrite.
5. **Read-only MQTT evidence collector** — prove exact capture and broker ACL isolation.
6. **Verifier cryptographic checks** — prove record/segment/checkpoint validation on fixed fixtures.
7. **Correlation + pinned trusted decoder** — prove journal/MQTT/ChirpStack/TimescaleDB lineage and mismatch classification.
8. **gateway_evidence.event_verification gate** — prove only verifier-owned verified results make v2 eligible.
9. **Fabric Adapter implementation** — only after the reviewed image/runtime exists and the external Fabric handoff is frozen.
10. **OpenBao/Fabric end-to-end attestation** — prove persisted seal verification, valid commit, unknown-result reconciliation, and idempotency.

For **initial commissioning**, prove only the minimum normal-path and trust-boundary checks needed to show the component works and cannot exceed its intended authority. Keep outage, process-loss, lease-expiry,

tamper/reorder/delete/checkpoint-conflict, and other fault matrices in Guide 3 / Phase 15. A deeper failure test is required early only when the specific boundary cannot be trusted without it, such as rejecting an unauthenticated evidence upload.

4.11 Current implementation status boundary

Current recorded state:

telemetry.fabric_outbox schema/ACL/immutability	-> commissioned / PASS
OpenBao 3-node KMS + audit	-> commissioned / PASS
SeaweedFS raw evidence infrastructure S0-S9	-> commissioned / PASS
gateway_evidence migration/HBA/CONNECT/six LOGIN identities	-> commissioned / PASS
PgBouncer evidence expansion	-> three-node ten-role PASS
Immutable cloud OCI + ingest/collector/verifier replicas	-> commissioned / PASS
Evidence PKI + collector MQTT identities/ACLs	-> commissioned / PASS
Shared anchor :443 SNI + Grafana evidence views	-> commissioned / PASS
Fabric adapter ULC-01	-> enabled continuous writer / PASS
Fabric adapter ULC-02	-> write-disabled; HA fencing/ownership acceptance pending
Gateway Rust writer/uploader + OpenWrt runtime	-> installed / real lineage PASS

The cloud and target Gateway-01 implementations are live rather than source-only claims. The OpenWrt evidence runtime, protected gateway identities/configuration, IPC permission guard, journal/uploader receipts, and one assembled EMU-01 physical lineage are proven. Preserve this accepted normal path and repeat target compilation/commissioning only after a relevant package/runtime change.

OpenBao audit closure and ULC-01 Fabric activation are complete. Preserve that audit boundary and the commissioned ULC-01 least-privilege writer. Keep ULC-02 fail-closed until its separate HA fencing/ownership acceptance gate passes.

4.12 One-sentence architecture summary

The gateway independently hash-chains what Concentratord observed, the cloud verifies that history against what MQTT and ChirpStack actually delivered and what the trusted decoder reconstructs from stored telemetry, and only the resulting verifier-owned `verified` evidence can be sealed by OpenBao and asynchronously committed to Hyperledger Fabric.

Supporting runbook: 06-replicated-ha-deployment-journey.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

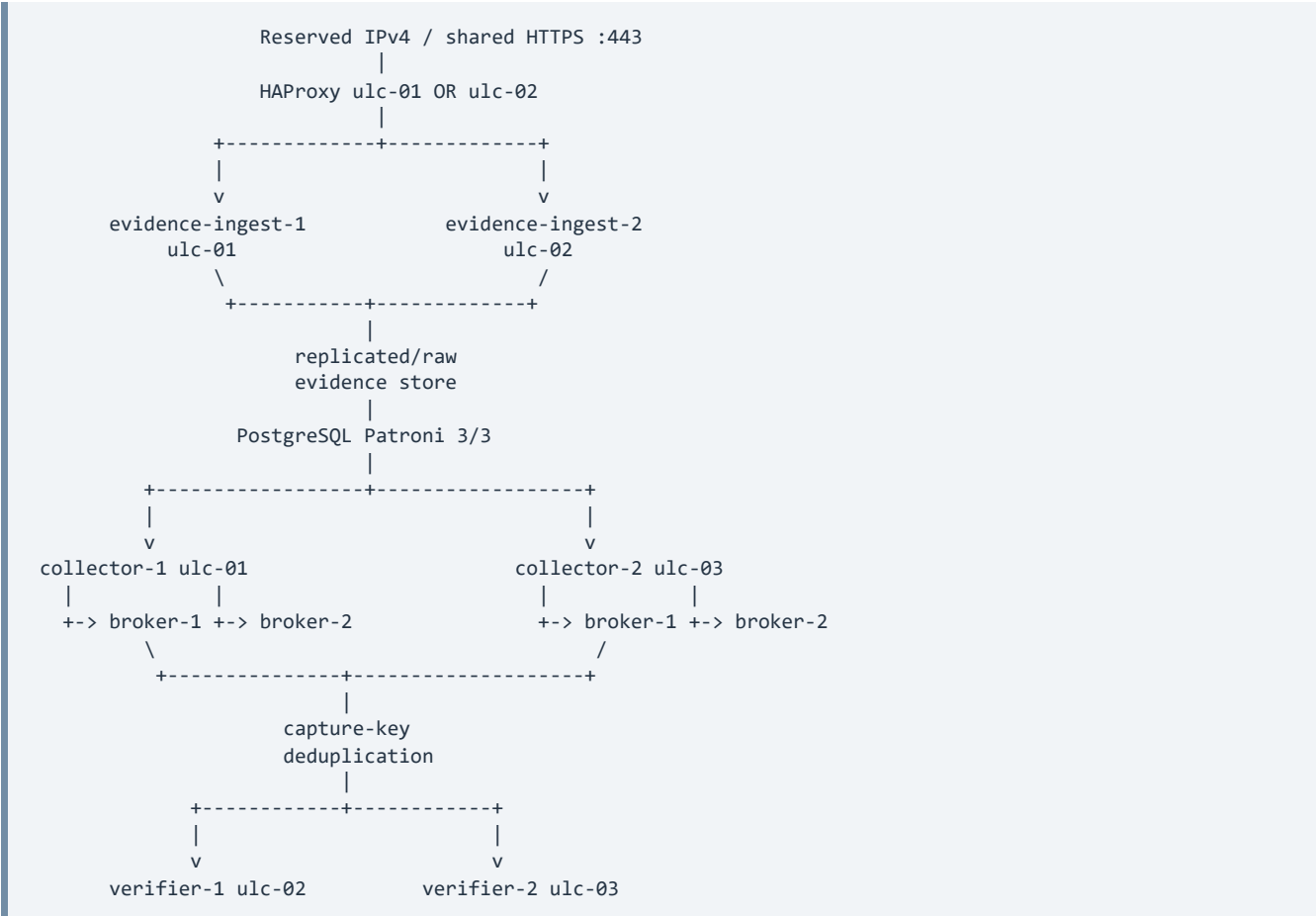
Replicated Gateway Evidence Services - Streamlined Deployment Journey

Status: LIVE COMMISSIONING IN PROGRESS. Source/build readiness, SeaweedFS infrastructure through the HAProxy boundary, the reviewed gateway_evidence migration, and the three-node evidence-aware PostgreSQL HBA are already live PASS. Production object-store helper S9, evidence workload LOGIN/PgBouncer closure, PKI/ingress, and service replicas remain. Preserve completed gates and resume only from the current failed boundary.

6.1 Goal

Build the evidence tier so one cloud service instance can disappear without losing an accepted evidence object or creating two authoritative verification results.

Target runtime shape after capacity validation:





The gateway journal/uploader remain single-device services because there is one physical gateway. Their resilience comes from crash-safe local state, independent MQTT delivery buffering, and cloud checkpoints; do not call them an HA pair.

6.2 Non-negotiable HA semantics

```

Ingest
  active/active
  same retry -> one accepted identity
  same identity + different digest -> security conflict

Collector
  active/active
  two replicas
  each replica -> persistent read-only session to broker-1 AND broker-2
  duplicate observations -> one deterministic capture_key_sha256

Verifier
  active/active
  two replicas
  FOR UPDATE SKIP LOCKED + expiring lease
  crash -> lease expires -> other worker reclaims

Trusted decoder
  stateless
  same immutable code/image digest on both verifier replicas

Raw storage
  one-Droplet loss cannot remove the only copy
  create-only/no-overwrite
  exact-byte SHA-256 recovery proof required

Database
  existing 3-node Patroni cluster is the authoritative metadata/state layer
  
```

6.3 Why the setup is split into guarded blocks

One giant script that mutates storage, PostgreSQL, HAProxy, PKI, three services, and Grafana makes failure recovery ambiguous. Instead, use one block per trust boundary. Keep initial commissioning deliberately small: prove each replica starts, then run one representative convergence/function check for the pair. Deep failure testing stays in Guide 3 / Phase 15.

Standard wrapper behavior:

```

preflight -> install/start replica 1 -> health -> install/start replica 2 -> health -> one
functional/convergence smoke -> evidence -> PASS
  
```

If a block fails after the canary passes, preserve that PASS and resume from the failed replica/check. Do not rerun the whole phase automatically.

Standard guarded one-block shell pattern

Every future live block should follow the same controlled child-shell shape below. This is a **structural template**, not a command to run now: a delivered live block must replace the marked body with reviewed, concrete commands and remove the template stop gate before the operator executes it.

```
sudo bash <<'EOF'
set -euo pipefail

# TEMPLATE SAFETY: a real reviewed block removes this guard.
echo 'TEMPLATE_ONLY=STOP'
exit 64

STEP='<BOUNDARY_NAME>'
RUN_ID="EVIDENCE-SVC-$(date -u +%Y%m%dT%H%M%SZ)"
EVIDENCE_DIR="/home/opsadmin/lorawan-ha-evidence/${RUN_ID}/<STEP_DIR>"
install -d -m 700 "$EVIDENCE_DIR"

fail() {
    printf 'STEP=%s\nRESULT=FAIL\nREASON=%s\n' "$STEP" "$*" >&2
    exit 1
}

pass() {
    printf '%s=PASS\n' "$1"
}

# -----
# 1. READ-ONLY PREFLIGHT
#   - previous PASS prerequisites
#   - current service/cluster/resource state
#   - exact target files/listeners free/expected
# -----

# -----
# 2. CANARY MUTATION
#   - minimum rollback copy for this boundary only
#   - validate candidate before reload/start
# -----

# -----
# 3. VERIFY CANARY
#   - health/restart/OOM
#   - permissions/ACLs
#   - functional positive + negative checks
# -----
pass REPLICAS_1

# -----
# 4. SECOND REPLICAS MUTATION
#   - execute only after REPLICAS_1 passed
# -----

# -----
# 5. VERIFY SECOND REPLICAS
# -----
pass REPLICAS_2

# -----
# 6. HA INVARIANT
#   - duplicate work converges safely
#   - durable state is shared/replicated as designed
#   - no unrelated service regression
# -----
pass HA_INVARIANT

# -----
# 7. SECRET-FREE EVIDENCE
#   - bounded command output/config hashes only
#   - never copy .env/private keys/passwords/tokens/SecretIDs
# -----
find "$EVIDENCE_DIR" -type d -exec chmod 700 {} +
```

```

find "$EVIDENCE_DIR" -type f -exec chmod 600 {} +
(
  cd "$EVIDENCE_DIR"
  find . -type f ! -name SHA256SUMS -print0 \
    | sort -z \
    | xargs -0 -r sha256sum > SHA256SUMS
  sha256sum -c SHA256SUMS
)

printf 'STEP=%s\n' "$STEP"
printf 'UTC=%s\n' "$(date -u +%Y-%m-%dT%H:%M:%SZ)"
printf 'REPLICA_1=PASS\n'
printf 'REPLICA_2=PASS\n'
printf 'HA_INVARIANT=PASS\n'
printf 'EVIDENCE_PATH=%s\n' "$EVIDENCE_DIR"
printf '<BOUNDARY_PASS_MARKER>=PASS\n'
EOF

```

Why this pattern is preferred:

- it keeps one logical deployment boundary in one copy/paste operation;
- replica 2 is unreachable if the canary fails because the child shell exits immediately;
- successful early gates can be preserved if a later verification fails;
- evidence generation is part of the block instead of a separate forgotten step;
- the block does not create another full backup cycle;
- the top-level login shell is not left in set -euo pipefail mode;
- future wrappers that call SSH must use ssh -n for no-input checks or redirect from an explicit payload file so SSH cannot consume the parent heredoc.

For each actual boundary in Sections 6.5-6.12, the project session should produce the concrete reviewed version of this block only when all required image names, paths, identities, ports, and rollback actions are known.

6.4 Journey evidence root

For live deployment use one protected run tree on the control host:

```

/home/opsadmin/lorawan-ha-evidence/EVIDENCE-SVC-<UTC>/
00-preflight/
01-storage-db/
02-pki-ingress/
03-ingest/
04-collector/
05-verifier-decoder/
06-observability/
07-normal-path/
SHA256SUMS

```

Directory mode 0700; evidence files 0600. Do not copy .env, private keys, database passwords, OpenBao tokens, SecretIDs, recovery material, or Fabric client keys into this tree.

6.5 Block 0 - artifact and capacity preflight

Mutation: none.

Require before any live evidence service is created:

```

reviewed source revision for all three server services
immutable image/executable digest for each
runtime UID/GID

```

```
health/readiness contract
configuration schema
trusted-decoder immutable digest + fixtures
v2 canonicalization vector
selected replicated evidence-storage technology
reviewed gateway_evidence migration/grants
reviewed shared-443 design
collector TLS/client-ID design for both brokers
rollback commands
```

The one-block preflight must inspect all three hosts for memory, disk, listeners, current containers, restart/OOM state, and the exact ports/mounts the candidate would consume. Do not start anything.

Pass marker:

```
EVIDENCE_REPLICATED_PREFLIGHT=PASS
```

6.6 Block 1 - replicated evidence storage + PostgreSQL migration

Mutation: storage namespace/backend plus one reviewed PostgreSQL migration only.

Guarded order inside one block:

1. re-prove Patroni 1 leader / 2 replicas and selected durable storage healthy
2. create the protected evidence namespace/bucket/path with create-if-absent policy
3. write one synthetic non-secret object and verify its exact SHA-256
4. apply the reviewed versioned `gateway\_evidence` migration once on the current primary
5. verify tables/constraints/indexes/lease columns/capture-key uniqueness and grants
6. verify both PostgreSQL replicas received the schema
7. delete only the synthetic storage probe if the backend contract permits test cleanup
8. capture hashes/metadata and PASS

Do not run a destructive storage failure here; Phase 15 owns actual member-loss testing.

Pass markers:

```
EVIDENCE_STORAGE_REPLICATION=PASS
EVIDENCE_DB_MIGRATION=PASS
```

6.7 Block 2 - evidence PKI + shared HTTPS ingress

Mutation: evidence certificates/identities and reviewed HAProxy shared-443 configuration only.

Canary-first order:

1. verify current ChirpStack HTTPS before change
2. verify anchor ownership/listeners
3. issue/stage evidence server certificate and per-gateway clientAuth identity without printing private material
4. validate SAN/EKU/chain/fingerprint/expiry/EUI mapping
5. build HAProxy candidate for ulc-01
6. haproxy -c before reload
7. activate canary
8. prove ChirpStack browser/API HTTPS still works without a client cert
9. prove evidence SNI route rejects no-cert/unknown-client
10. prove valid evidence client reaches only evidence backend
11. only then repeat the reviewed config on ulc-02
12. prove both anchor candidates equivalent
13. capture config hash/cert metadata and PASS

The exact SNI/client-cert method remains blocked until the reviewed HAProxy 2.8 configuration exists. Do not globally require a gateway certificate on ordinary ChirpStack HTTPS.

Pass marker:

```
EVIDENCE_PKI_INGRESS=PASS
```

6.8 Block 3 - two active/active ingest replicas

Mutation: deploy ingest-1 then ingest-2.

Frozen target placement:

```
ulc-01 -> evidence-ingest-1  
ulc-02 -> evidence-ingest-2
```

One block may do both replicas only with this gate sequence:

1. verify storage + DB + PKI/ingress PASS markers and stage the identical immutable image/config on both hosts
2. install distinct ingest-1 credential; start ingest-1; verify health/runtime UID/listener/resource state
3. install distinct ingest-2 credential; start ingest-2; verify the same health gates
4. send one approved synthetic checkpoint/segment fixture through the normal ingress/backend path
5. repeat the exact fixture once and prove one raw object + one metadata identity remain
6. prove object SHA matches metadata and both replicas remain healthy
7. capture evidence and PASS

Pass marker:

```
EVIDENCE_INGEST_REPLICATED=PASS
```

6.9 Block 4 - two active/active MQTT evidence collectors

Mutation: dedicated collector identities/ACLs and collector-1/2 runtime.

Frozen target placement:

```
ulc-01 -> collector-1  
ulc-03 -> collector-2
```

Each collector has **two broker sessions**:

```
collector-N/broker-1 -> 10.104.0.2:8884, TLS identity mqtt.internal.lorawan.com  
collector-N/broker-2 -> 10.104.0.4:8884, TLS identity mqtt.internal.lorawan.com
```

Use distinct client IDs for all four sessions and persistent subscriptions. ACL is read-only as923/gateway/+/event/#; publish and command topics are denied.

Guarded order:

1. verify both broker backends, TLS identity, and the reviewed read-only ACL configuration
2. stage/start collector-1 and prove its two persistent broker sessions
3. stage/start collector-2 and prove its two persistent broker sessions
4. feed one approved non-RF commissioning fixture through the reviewed safe input path when available
5. prove duplicate observations converge to one `capture\_key\_sha256` and one logical raw-object identity

```
6. capture the four client/session health states without secrets
7. PASS
```

Do not interpret the duplicate observation by collector-1/2 as two gateway events.

Pass marker:

```
EVIDENCE_MQTT_COLLECTOR_REPLICATED=PASS
```

6.10 Block 5 - two verifier workers + trusted decoder

Mutation: verifier-1/2 runtime only; no OpenBao/Fabric permission.

Frozen target placement:

```
ulc-02 -> verifier-1 + trusted decoder
ulc-03 -> verifier-2 + trusted decoder
```

Both must report the exact same decoder digest/version before either can process work.

Guarded order:

```
1. prove one fixed valid decoder vector on both replicas and require identical decoder digest/version
2. start verifier-1 and verify health + least-privilege DB configuration
3. start verifier-2 and verify the same health/configuration gates
4. queue one fixed pending verification fixture
5. prove exactly one worker owns/processes it and the terminal state/reason/digests match the fixture
6. prove no duplicate terminal row/work item appeared and both replicas remain healthy
7. PASS
```

Actual host/process failure timing, lease-expiry/reclaim, SKIP LOCKED distribution under concurrent load, and tamper/reorder/delete/checkpoint-conflict fixtures belong to Guide 3 / Phase 15. They are not required to get the replicated verifier pair installed and working.

Pass markers:

```
EVIDENCE_VERIFIER_REPLICATED=PASS
TRUSTED_DECODER=PASS
```

6.11 Block 6 - read-only evidence observability

Mutation: Grafana provisioning/read-only views only.

Add panels/queries for:

```
replica health ingest-1/2
collector-1/2 broker-session health
verifier-1/2 worker/lease state
latest checkpoint age per gateway
pending verification age
verified/evidence_gap/integrity_failure counts
collector unmatched count
trusted-decoder mismatch count
raw-store replication/member health
v2 outbox rows blocked on verification
```

Telemetry freshness and evidence freshness remain different panels.

Pass marker:

```
EVIDENCE_OBSERVABILITY=PASS
```

6.12 Block 7 - real normal-path lineage

This step waits for the physical gateway and reviewed server runtimes.

One real staging uplink must produce exactly one lineage:

```
journal record
-> closed segment/checkpoint
-> replicated evidence ingest
-> one logical MQTT capture despite collector replication
-> accepted ChirpStack application event
-> trusted decoder result
-> TimescaleDB row
-> one event_verification row = verified
```

Then and only then may the matching v2 outbox row become eligible for the Fabric Adapter.

Pass marker:

```
GATEWAY_EVIDENCE_V2_NORMAL_PATH=PASS
```

6.13 What gets written to Markdown after every block

After every operator block, record only verified facts:

```
UTC/date
boundary name
hosts/replicas touched
immutable image/executable hashes
config/migration hash where relevant
non-secret certificate metadata where relevant
resource snapshot
named PASS markers
any harness defect and exact resume point
what was deliberately not repeated
next boundary
```

Update:

```
deployment/server/integrations/gateway-integrity/<relevant-guide>.md
deployment/server/cloud-production/00-build-execution-log.md
deployment/server/cloud-production/00-current-server-continuation-checkpoint.md
```

If a block partially passes, write the successful gates immediately and resume from the first failed gate. Do not discard valid evidence and do not rerun successful replicas only for ceremony.

6.13A Live execution checkpoint — 2026-08-31

This section is the authoritative resume point after the interrupted documentation period. Do not rerun completed boundaries merely to reconstruct evidence.

Raw evidence infrastructure

```
SeaweedFS release      4.41 / commit de34a1a87
SeaweedFS image
chrislusf/seaweedfs:4.41@sha256:43b768cd62b00d132439cda881b93fd1adebf1b315e996e794087743821d771d
metadata-etcd image
quay.io/coreos/etcd:v3.5.15@sha256:0934690612905554eb61ddefb9faaaecb47c2f6931dbb453e694358092ee8990
metadata-etcd ports    12379 / 12380
master                 19333 HTTP / 19334 gRPC
volume                 18082 HTTP / 18083 gRPC
filer                  18888 HTTP / 18889 gRPC
raw S3                  127.0.0.1:18333
S3 TLS boundary        node VPC :18443
replication             010 = one additional copy, different rack, same sgp1 DC
```

SEAWEEDFS_CORE_3_NODE=PASS and SEAWEEDFS_REPLICATION_010_EMPIRICAL=PASS are authoritative. The retained placement fixture is 3,01e3ab96f3, size 89, SHA-256 bf981516163ff1e35d6315213458423860be84f0b7fe74269ac8d780577bb5b, with copies on two distinct Droplet racks. The lorawan-evidence bucket is configured with replication 010 and volume growth count 2.

Runtime S3 identities are installed for ingest, collector, and verifier. Ingest/collector have read/list/write; verifier has read/list only. The active s3.json is identical on all three hosts, SHA-256 310aa8b74145256bae9e15f759bacfc37d590a5b54c08c348c38ea7e0c6371f8, protected as 0640 root:1000. The earlier leaked S3 credentials were retired and must never be reused.

Internal object-store PKI is live. CA SHA-256 is c1dedc8cc6b58217e955cf763868b429dacdd933bbe7d9ffed147122e9d013fd; logical endpoint is evidence-objects.internal.lorawan.com:18443. HAProxy on ulc-01/02/03 accepts GET/HEAD and only conditional PUT with exactly one If-None-Match: *; other methods are 405 and bad/unconditional PUT is 428. Raw Seaweed S3 remains loopback-only. HAPROXY_EVIDENCE_S3_3_NODE=PASS is authoritative. Retained HAProxy fixtures exist on all three nodes and must not be deleted merely to tidy commissioning evidence.

S9 COMPLETE / PASS on 2026-09-01. The locked production gateway-evidence-ingest helper executed objectstore-contract-write through https://evidence-objects.internal.lorawan.com:18443; the retained object and race fixture were then verified cross-host from ulc-03. Record OBJECTSTORE_CONTRACT=PASS, SEAWEEDFS_S9_WRITE=PASS, SEAWEEDFS_S9_CROSS_HOST_VERIFY=PASS, and EVIDENCE_OBJECTSTORE=PASS. Retain the commissioning fixtures and do not repeat S0-S9 without a storage change or failure.

gateway_evidence database migration

The live migration is COMPLETE / PASS. Repository source hashes remain:

```
001_gateway_evidence.sql      bf2a1e3188cf67107872c425064d55fb476d7ea58855510b154f7e869795a8b9
001_gateway_evidence.verify.sql e08112fd48cdb6be058f40487ac7fff4d4b60a9647f2e1750cf1ebbd974ed4ae
```

The reviewed terminal transport copies that were actually executed had hashes cd03a10f39dc4e780be5f7c7718596816a199dfaf118d7bd9c3f1c5ce4e1a630 and fc57361eda4fc7fb20e3148eed421e2f653f38dfc25e00e6dee07dba59adc040; preserve that distinction instead of claiming byte identity with the repository files.

Pre-migration backup on current leader ulc-01 is /root/backups/evidence-db-pre-migration/20260831T074754Z/lorawan_telemetry.dump, SHA-256

f9564ffaf8e65e021fa025cd45e70c9472608bfa7b6e4912cc568cc773aa21de, size 79962, with pg_restore --list PASS. The Timescale continuous_agg circular-FK warning was informational because this was a full custom-format dump, not a data-only dump.

EVIDENCE_DB_MIGRATION=PASS, EVIDENCE_DB_PRIMARY_VERIFY=PASS, and EVIDENCE_DB_3_NODE_REPLICATION=PASS are authoritative. The migration created four gateway_evidence base tables, three nullable telemetry provenance columns, six relevant indexes, the monotonic checkpoint trigger, and three passwordless NOLOGIN authority roles:

```
gateway_evidence_ingestor
gateway_evidence_collector
gateway_evidence_verifier
```

Evidence PostgreSQL HBA rollout

The HBA extension is COMPLETE / PASS on all three Patroni members. The policy remains exactly 20 rules: no broad rule was added. Instead, the existing three hostssl lorawan_telemetry ... /32 scram-sha-256 rules now use this user selector:

```
telemetry_writer,telemetry_reader,fabric_adapter,+gateway_evidence_ingestor,+gateway_evidence_collector,+gateway_evidence_verifier
```

All three live/persistent HBA lists have SHA-256

a943358a884249aaae74b663a81fa6dde2d7c98deeb31f93def8e5bb4aa729f1, zero parse errors, and the two final reject rules remain intact. Rollout order was ulc-03 canary/persist -> ulc-02 canary/persist -> ulc-01 leader canary/fresh-auth/persist. No Spilo restart/recreate or Patroni switchover occurred.

Three harness defects were corrected during the rollout and must not be reintroduced:

1. Patroni reload is asynchronous; poll pg_hba_file_rules for convergence instead of sleep 3.
2. OLD_USERS is a prefix of NEW_USERS; compare the parsed HBA user field exactly, not substring containment.
3. pg_stat_replication.client_addr is inet; use host(client_addr), not client_addr::text, for exact address matching.

Before leader persistence, brand-new physical-replication IDENTIFY_SYSTEM sessions were created from ulc-02 source 10.104.0.4 and ulc-03 source 10.104.0.8 using standby, SCRAM, and sslmode=verify-full. Both returned system identifier 7676855802088521796, timeline 3, LSN 0/68000000. FRESH_REPLICATION_AUTH_TO_ULC01=PASS is authoritative.

Evidence database credentials + PgBouncer rollout

The planned identities are:

```
evidence_ingest_ulc01    -> gateway_evidence_ingestor
evidence_ingest_ulc02    -> gateway_evidence_ingestor
evidence_collector_ulc01 -> gateway_evidence_collector
evidence_collector_ulc03 -> gateway_evidence_collector
evidence_verifier_ulc02  -> gateway_evidence_verifier
evidence_verifier_ulc03  -> gateway_evidence_verifier
```

Historical issuance note: the first evidence_ingest_ulc01 login exposed the missing group-level database CONNECT grant, producing permission denied for database "lorawan_telemetry". That stop was correctly attributed to the database ACL layer rather than HBA/TLS/SCRAM and was fixed without rotating the existing credential. The PASS results below supersede that intermediate stop.

CONNECT authority correction: PASS on 2026-09-01. After two harness-only attempts that stopped before mutation, the corrected gate re-proved pre-state t|t|t|t|f|f|f|f, granted CONNECT ON DATABASE

lorawan_telemetry only to gateway_evidence_ingestor, gateway_evidence_collector, and gateway_evidence_verifier, and proved post-state `t|t|t|t|t|t|t|t` on all three PostgreSQL members while PUBLIC remained denied. The existing evidence_ingest_ulc01 password was not regenerated; its fresh direct SCRAM + verify-full session succeeded and its exact least-privilege role structure remained intact.

Evidence LOGIN issuance: COMPLETE / PASS on 2026-09-01. Safe-resume preserved evidence_ingest_ulc01, created only the five absent roles, and closed all six with unique protected SCRAM credentials, one exact intended NOLOGIN authority membership, inherited database CONNECT, direct verify-full authentication, and three-node catalog state `6|6|6|6|6|6`. PgBouncer remained unchanged at four entries with zero evidence identities throughout.

PgBouncer three-node evidence expansion: COMPLETE / PASS on 2026-09-01. The final live inspection found ulc-01, ulc-02, and ulc-03 already converged on the same ten-entry/six-evidence SCRAM userlist, mode 0640 root:postgres, size 1582 bytes, SHA-256 665f1592c96ca276681a454b9cbcd6ab8ab0cbfb4594b8ddd443239db58df391. The prior four-role file is SHA-256 e159c1c3e408eb091629083a81738c730bb8d2ab3c07ecbc6aaf109c51b9d5e8. Rollback directories are /etc/pgbouncer/evidence-userlist-rollout-20260901T004328Z on ulc-01, ...T020032Z on ulc-02, and ...T020244Z on ulc-03. The ulc-02/ulc-03 rollback sets prove byte-identical before/candidate verifiers for all four pre-existing roles. PgBouncer remains active/enabled with PIDs 1698991, 1512906, and 1690020, matching their pre-expansion recorded PIDs and therefore proving reload without process restart.

A fresh strict verify-full acceptance used three commissioned evidence roles against physical endpoints 10.104.0.2:6432, .4:6432, and .8:6432. All three authenticated, negotiated TLS 1.3 / TLS_AES_256_GCM_SHA384, and reached writable lorawan_telemetry on current Patroni primary 10.104.0.2. PgBouncer expansion is no longer a resume item. SeaweedFS S9, PKI/collector identities, immutable OCI rollout, replicated service readiness, shared-443, and Grafana evidence views subsequently passed; the cloud journey is now EVIDENCE_CLOUD_RUNTIME=PASS.

6.14 Final commissioning rule

```
replicated process != replicated truth
```

HA is accepted only when durable state makes duplicate execution safe and one-instance loss recoverable:

Ingest replicas	-> idempotent stable identities
Collector replicas	-> deterministic capture key
Verifier replicas	-> durable leases
Trusted decoder replicas	-> identical immutable code
Raw evidence store	-> cross-host durable bytes
PostgreSQL	-> Patroni HA
OpenBao	-> Raft HA
Fabric adapter	-> lease-safe dual workers

No component may claim replication merely because replicas: 2 appears in Compose.

6.15 Current cloud commissioning result - 2026-09-01

The complete server-side journey is PASS: raw-object storage S0-S9, database roles/PgBouncer, Evidence PKI, collector mTLS identities/ACLs, replicated evidence services, disabled Fabric standbys, shared-443 mTLS passthrough, MQTT witness capture, and read-only Grafana checkpoint/verification panels. The subsequent Gateway-01 target-package and real assembled EMU-01 physical-lineage gate is also PASS; preserve this normal path and move remaining work to sensor test execution, provider failover acceptance, or deliberate Fabric activation.